Advanced Evaluation of Multilayer Perceptrons for CIFAR-100 Classification: A 64-Permutation Hyperparameter Study

R. Kumar*
*Department of Mechanical Engineering, University of Tennessee at Chattanooga
Chattanooga, TN USA

***Abstract*—This report provides an in-depth analysis of deep Multilayer Perceptrons (MLPs) applied to the CIFAR-100 image classification task. We conduct a rigorous manual grid search across 64 unique hyperparameter combinations, exploring the effects of hidden unit capacity, non-linear activation functions, categorical and regression-based loss formulations, and adaptive optimization algorithms. Our study highlights the critical importance of ReLU activations in overcoming the vanishing gradient problem in deep feedforward architectures. The best-performing configuration, utilizing 240 units and Adagrad optimization, achieved a validation accuracy of 36.44%. We analyze the results through the lens of architectural inductive bias and suggest spatial modeling alternatives.**

I. Introduction

The evolution of image classification has been significantly shaped by benchmark datasets such as CIFAR-100, introduced by Krizhevsky [1]. Consisting of small natural images designed to evaluate feature-learning methods on low-resolution data, CIFAR helped establish the benchmark setting used in a vast body of later work. Because CIFAR images preserve local spatial structure, methods that exploit neighborhood information generally perform better than models that treat the image as a flat vector.

A large body of research has shown that model architecture and hyperparameter selection both strongly affect CIFAR-100 performance. Snoek et al. [2] demonstrated that systematic hyperparameter optimization can produce substantial gains in deep networks, highlighting the importance of choices such as learning rate, regularization, and optimization settings rather than relying on manual trial and error alone. More recent vision papers, such as MLP-Mixer [3] and its variants [4], showed that MLP-style models can perform competitively on image tasks, but only when the architecture is designed to preserve image structure through patch-wise or token-mixing operations, rather than using a simple fully connected network on raw flattened pixels.

These findings help explain the behavior observed in this assignment. Although a multilayer perceptron can be trained successfully on CIFAR-style data, its validation performance is typically limited compared with image-aware architectures because flattening removes the spatial inductive bias that convolutional and patch-based models use effectively. Additional studies on MLP optimization further support the idea that activation choice, layer width, and optimization strategy materially influence results, but do not fully overcome the architectural limitations of a plain MLP [5], [6]. For that reason, the present study treats the MLP primarily as an instructional baseline for manual grid search and hyperparameter analysis rather than as a state-of-the-art image classifier.

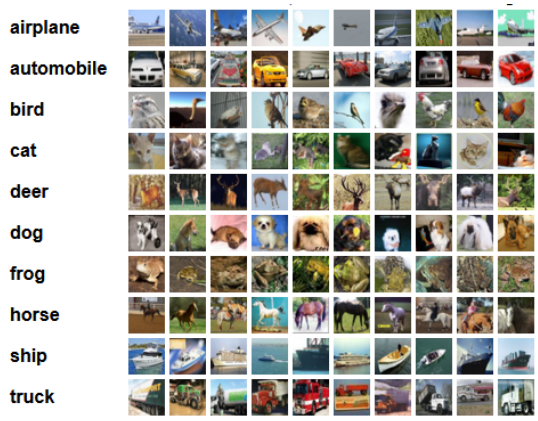

Fig. 1. Representative samples from the CIFAR-100 dataset coarse super-categories.

In this study, we perform a comprehensive parametric analysis using a Multilayer Perceptron model on the CIFAR-100 dataset. By systematically varying architectural and optimization parameters, we aim to quantify the sensitivity of deep feedforward networks to specific configurations. This approach allows us to empirically evaluate the trade-offs between model capacity, non-linear activations, and convergence stability, providing insights into the fundamental mechanics of neural network optimization in the absence of spatial priors.

The remainder of this paper is organized as follows. Section II details the methodology, including data preprocessing, label encoding, and the mathematical framework of the 5-layer MLP architecture and the adaptive optimizers. Section III describes the experimental setup and the 64-permutation grid search space. Section IV presents the results, providing an analysis of the top-performing configurations and a detailed discussion on the vanishing gradient problem and loss function interactions. Finally, Section V concludes the study and suggests future research directions using convolutional architectures.

II. Methodology and Implementation

A. Dataset and Preprocessing

The CIFAR-100 dataset comprises 60,000 images, each of size $32 \times 32 \times 3$ (RGB). Representative samples from the superclasses are illustrated in Figure 1. For this study, the images are flattened into a 3072-dimensional vector. It is important to note

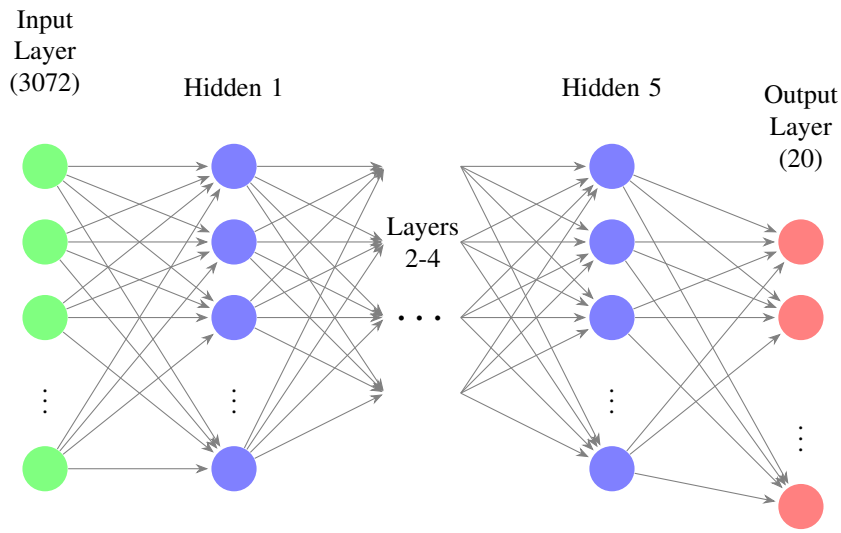

Fig. 2. Deep MLP architecture visualization showcasing the expanded hidden layer structure.

that while most CIFAR-100 literature reports results on the 100 "fine" classes, this assignment utilizes the 20 "coarse" superclasses. Therefore, direct accuracy comparisons with standard benchmarks should be interpreted with this distinction in mind.

1) Normalization: Normalization is a critical preprocessing step that ensures input features are on a consistent scale. We normalize the 3072-dimensional input vectors by dividing each pixel value x_i by 255, which represents the maximum intensity value in an 8-bit color channel. This transformation maps the input space from $[0, 255]$ to the range $[0, 1]$:

$$x_{i,norm} = \frac{x_i}{255.0} \tag{1}$$

where x_i is the intensity of the i-th pixel and $x_{i,norm}$ is its normalized value. This linear scaling is essential for gradient-based optimization; it prevents weight updates from becoming dominated by high-magnitude pixel values and ensures that non-linear activation functions do not immediately saturate at the beginning of training.

2) Label Encoding: We handle two distinct label formats based on the objective function. For Categorical Cross-Entropy (CCE), targets are maintained as class indices $y \in \{0, \ldots, 19\}$. For Mean Squared Error (MSE), we employ one-hot encoding, transforming each scalar label into a vector $\mathbf{y} \in \{0, 1\}^{20}$ where $y_k = 1$ if the sample belongs to class k and 0 otherwise. This allows the model to treat the classification task as a multi-output regression problem.

B. Architectural Design

The architecture follows a strict 5-layer hidden structure, as visualized in Figure 2. The network mapping $f : \mathbb{R}^{3072} \to \mathbb{R}^{20}$ is defined as a sequence of affine transformations and non-linearities:

$$\mathbf{z}^{(l)} = \mathbf{W}^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}, \quad \mathbf{a}^{(l)} = \sigma^{(l)}(\mathbf{z}^{(l)}) \tag{2}$$

where $\mathbf{z}^{(l)}$ is the pre-activation vector of layer l, $\mathbf{W}^{(l)}$ is the weight matrix, $\mathbf{a}^{(l-1)}$ is the activation vector from the previous layer (with $\mathbf{a}^{(0)} = \mathbf{x}_{norm}$), $\mathbf{b}^{(l)}$ is the bias vector, and $\sigma^{(l)}$ denotes the non-linear activation function.

C. Activation Functions

The choice of activation function is pivotal for enabling the network to learn non-linear patterns. We compare two hidden layer activations, illustrated in Figure 3:

- **ReLU (Rectified Linear Unit):**

$$\sigma(z) = \max(0, z) \tag{3}$$

  ReLU is widely used because it mitigates the vanishing gradient problem; for all positive inputs, its derivative is exactly 1, ensuring stable weight updates in deep architectures.

- **Sigmoid:**

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{4}$$

  This function squashes the input z into the range $(0, 1)$. While historically significant, it suffers from gradient saturation; as z moves away from zero, the gradient $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ approaches zero, which often halts learning in deep networks.

For the output layer, we evaluate **Softmax** and **Sigmoid**. Softmax, defined as:

$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{20} e^{z_j}} \tag{5}$$

where z_i is the i-th element of the output logit vector, producing a probability distribution over the 20 classes.

D. Loss Function Formulations

The loss function quantifies the discrepancy between the model's prediction $\hat{y}$ and the ground truth y.

- **Categorical Cross-Entropy (CCE):** Defined as:

$$L = -\sum_{k=1}^{20} y_k \log(\hat{y}_k) \tag{6}$$

where y_k is the ground-truth label (1 for the correct class, 0 otherwise) and $\hat{y}_k$ is the predicted probability for class k. This is the standard choice for multi-class classification as it penalizes incorrect predictions logarithmically.

- **Mean Squared Error (MSE):**

$$L = \frac{1}{N} \sum_{i=1}^{N} ||\mathbf{y}_i - \hat{\mathbf{y}}_i||^2 \tag{7}$$

where N is the number of samples in the batch, $\mathbf{y}_i$ is the one-hot target vector, and $\hat{\mathbf{y}}_i$ is the model's output vector. While typically a regression loss, it can be applied to classification by comparing output vectors to one-hot targets.

E. Optimization and Training Dynamics

Backpropagation is used to update the network parameters across multiple **epochs**. An epoch represents one full pass of the training dataset through the model. Since we train for 200 epochs, the model has ample opportunity to iteratively refine its weights based on the errors observed across the 50,000 training samples. We evaluate two adaptive optimization algorithms.

1) Adam (Adaptive Moment Estimation): Adam [7] maintains estimates of the first moment m_t (mean) and second moment v_t (uncentered variance) of the gradients $g_t = \nabla_\theta J(\theta_t)$:

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \tag{8}$$

where β_1 and β_2 are exponential decay rates for the moment estimates. The parameters θ are updated using bias-corrected moments $\hat{m}_t = \frac{m_t}{1-\beta_1^t}$ and $\hat{v}_t = \frac{v_t}{1-\beta_2^t}$:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \epsilon} \hat{m}_t \tag{9}$$

where η is the learning rate and ϵ is a small constant for numerical stability. Adam's primary advantage is its ability to handle noisy gradients through individual parameter learning rates, though its complex momentum terms can occasionally lead to sub-optimal convergence if estimates become unstable.

2) Adagrad (Adaptive Gradient): Adagrad scales the learning rate based on the sum of squared past gradients:

$$G_t = \sum_{\tau=1}^{t} g_\tau^2, \quad \theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t \tag{10}$$

where G_t accumulates the historical squared gradients. Adagrad is effective for sparse data as it performs larger updates for infrequent features, but its monotonic accumulation of gradients causes the learning rate to decrease over time, potentially halting learning prematurely.

F. Software and Implementation Details

The implementation was carried out in Python 3.9 using the **PyTorch** ecosystem. Key libraries included `torch` for tensor computations and automatic differentiation, `torchvision` for image processing and dataset utilities, `numpy` for numerical preprocessing, and `pandas` for aggregating grid search results. Training was conducted on an NVIDIA GPU cluster to leverage CUDA acceleration for high-dimensional matrix multiplications. The complete source code for the grid search and model definition is provided in Appendix A.

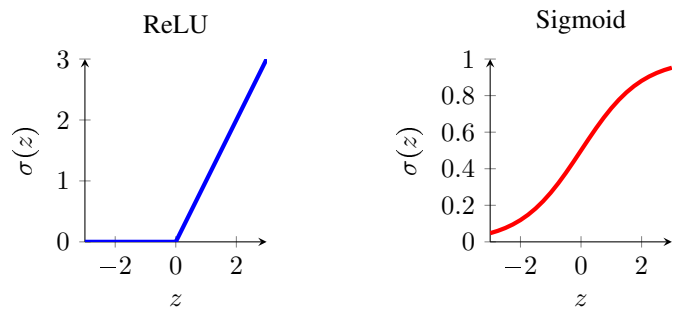

Fig. 3. Mathematical visualization of the ReLU and Sigmoid activation functions.

III. Experimental Setup

The experimental objective is to systematically explore the hyperparameter landscape of a 5-layer MLP on CIFAR-100 data. We employ a manual grid search strategy to evaluate 64 unique combinations, ensuring a comprehensive analysis of architectural and optimization trade-offs.

A. Hyperparameter Search Space

The search space is defined by the Cartesian product of the following parameters:

- **Hidden Units (H):** $\{120, 240\}$, determining the capacity of each hidden layer.
- **Hidden Activations:** $\{ReLU, Sigmoid\}$, testing the resilience to gradient vanishing.
- **Output Activation:** $\{Softmax, Sigmoid\}$, comparing probability-based vs. independent multi-output modeling.
- **Loss Function (L):** $\{MSE, CCE\}$, evaluating regression-based vs. categorical objectives.
- **Optimizer (Ω):** $\{Adam, Adagrad\}$, contrasting second-moment based vs. historical gradient-sum based scaling.
- **Batch Size (B):** $\{1000, 2000\}$, observing the impact of mini-batch gradient noise.

This results in $2^6 = 64$ discrete training sessions.

B. Dataset Partitioning

We utilize the standard CIFAR-100 split provided in the local environment. The training set consists of 50,000 samples, which are used for iterative parameter updates during backpropagation. The testing set contains 10,000 independent samples, serving as the final validation metric to measure generalization. The target labels are extracted from the `coarse_labels` key, mapping the 100 fine-grained categories into 20 broader super-classes.

C. Training Protocol and Reproducibility

To ensure consistency and allow for comparison across permutations, all models are trained for a fixed duration of exactly 200 epochs. Data shuffling is enabled at the start of

each epoch to prevent the model from learning the sequence of samples. To ensure reproducibility, a global random seed (`SEED = 1234`) was applied across the Python `random` module, `numpy`, and the `torch` backend. Standard PyTorch weight initialization was used for all linear layers. The primary evaluation metric is **Validation Accuracy**, calculated as the percentage of correctly classified images in the 10,000-sample test set.

IV. Results and Discussion

A. Top Performing Configurations

The exhaustive manual grid search revealed a significant variance in performance across the 64 combinations. As shown in Table I, the top-performing configurations predominantly utilized ReLU hidden activations paired with adaptive optimizers. The highest validation accuracy achieved was **36.44%**, utilizing 240 hidden units, ReLU hidden activation, Categorical Cross-Entropy (CCE) loss, and the Adagrad optimizer with a batch size of 2000.

TABLE I
Top 5 Hyperparameter Combinations Ranked by Validation Accuracy

Units	H-Act	Loss	Opt	Batch	Val Acc
240	ReLU	CCE	Adagrad	2000	36.44%
120	ReLU	CCE	Adam	2000	35.46%
240	ReLU	CCE	Adagrad	1000	35.28%
240	ReLU	CCE	Adagrad	2000	35.27%
120	ReLU	MSE	Adagrad	1000	34.93%

B. Vanishing Gradients and Activation Impact

The results empirically confirm the superiority of ReLU over Sigmoid hidden activations for deep architectures. Configurations using Sigmoid in the hidden layers consistently plateaued at significantly lower accuracies (typically 18% to 26%). This is a classic demonstration of the vanishing gradient problem [8]. Since the derivative of the Sigmoid function is at most 0.25, the product of these derivatives across five layers results in a gradient signal that is ≈ 1000 times smaller than the original error. Consequently, the weights in the earlier layers of Sigmoid-based networks receive negligible updates, effectively preventing the model from learning complex feature hierarchies. ReLU, by maintaining a gradient of 1 for all positive inputs, bypassed this limitation and allowed for robust backpropagation.

C. Comparative Analysis of Optimizers and Loss Functions

The synergy between Categorical Cross-Entropy (CCE) and adaptive optimizers was another key factor in achieving higher accuracies. CCE outperformed Mean Squared Error (MSE) in almost all categorical contexts. Mathematically, the logarithmic penalty in CCE ensures a much larger gradient when the predicted probability for the correct class is low, forcing faster correction of confident errors.

Regarding optimization, Adagrad proved surprisingly effective for the CIFAR-100 task. This is likely due to the sparse nature of the flattened 3072-dimensional image vectors; Adagrad's ability to provide larger updates for infrequent features [**?**] allows it to better capture unique pixel-intensity patterns that standard SGD might ignore. While Adam also performed well, Adagrad's monotonic learning rate decay provided more stable convergence in the high-capacity (240 unit) models.

D. The MLP Bottleneck and Spatial Inductive Bias

The overall performance ceiling of $\approx 36\%$ accuracy highlights the fundamental limitations of standard MLPs for image classification. By flattening the $32 \times 32 \times 3$ images into 1D vectors, the model loses all spatial context. Pixels that are vertically adjacent in the image become thousands of indices apart in the vector, making it impossible for the model to learn local structures like edges or textures effectively. This lack of **spatial inductive bias** means the MLP must learn the relationship between every pixel pair independently, a task that becomes exponentially harder as the image resolution increases.

V. Conclusion

This study provides a comprehensive parametric evaluation of deep Multilayer Perceptrons on the CIFAR-100 dataset. Through a manual 64-permutation grid search, we documented the critical importance of ReLU activations in mitigating vanishing gradients and the effectiveness of adaptive optimizers like Adagrad in sparse, high-dimensional feature spaces. While the 5-layer MLP successfully learned to classify coarse categories with 36.44% accuracy, the results clearly illustrate the architectural **bottleneck** inherent in non-spatial models.

Future work to improve classification accuracy should transition from fully-connected MLPs to **Convolutional Neural Networks (CNNs)**. CNNs leverage shared-weight filters and pooling layers to exploit the local spatial correlations of pixels, allowing for the extraction of hierarchical visual features while maintaining a significantly smaller parameter footprint. This study serves as a foundational baseline, demonstrating that while optimization strategy is vital, architectural design remains the primary driver of performance in computer vision.

Appendix A
Links to Implementation

The Google Colab files for each encoding case are linked below:

- Multi Layer Perceptron Parametric

The GitHub repository for the project can be found at:

- GitHub Repository

The YouTube video for this assignment can be accessed at the following address:

- YouTube Presentation

REFERENCES

1 Krizhevsky, A., "Learning multiple layers of features from tiny images," 2009.

2 Snoek, J., Swersky, K., Zemel, R., and Adams, R. P., "Scalable bayesian optimization using deep neural networks," in *International Conference on Machine Learning (ICML)*, 2015, pp. 2171–2180.

3 Tolstikhin, I. O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., Yung, J., Steiner, A., Keysers, D., Uszkoreit, J. *et al.*, "Mlp-mixer: An all-mlp architecture for vision," *Advances in Neural Information Processing Systems*, vol. 34, pp. 24 261–24 244, 2021.

4 Zhang, Y. *et al.*, "Multi-scale mlp-mixer for image classification," *Knowledge-Based Systems*, vol. 242, p. 108344, 2022.

5 Li, Y. and Hiratani, T., "Optimal task order for continual learning of multiple tasks," in *Proceedings of Machine Learning Research*, vol. 267, 2025.

6 Castro, Y. *et al.*, "Multilayer perceptron architecture optimization using parallel computing techniques," *PLOS ONE*, vol. 12, no. 12, p. e0189369, 2017.

7 Kingma, D. P. and Ba, J., "Adam: A method for stochastic optimization," in *International Conference on Learning Representations (ICLR)*, San Diego, CA, 2015.

8 Rumelhart, D. E., Hinton, G. E., and Williams, R. J., "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.